

TennisProject 系統架構設計與工作預定

版本：v1.0

更新日期：2026-04-22

適用範圍：目前 `/home/ct/TennisProject` 程式碼與部署方式

1. 文件目的

本文件整合兩件事：

1. 依照目前程式實作整理系統架構設計，作為開發與溝通基準。
2. 將現有零散待辦整理成可執行的工作預定，方便後續排程與驗收。

本文件偏向「實作導向」而非純提案文件，因此內容以目前 repo 已存在的模組、流程與部署方式為主。

2. 系統目標

TennisProject 的核心目標是將一段網球比賽影片，轉成可查詢、可視覺化、可追溯的分析結果，並提供 AI 問答能力。

主要輸出如下：

- 標注影片 `analysis.mp4`
- 結構化分析結果 `analysis.json`
- 回合縮圖 `thumbs/`
- 聊天歷史與分析紀錄

3. 系統範圍

目前系統範圍包含：

- 前端上傳、播放、分析進度與結果視覺化
- 後端影片管理、分析任務啟動、聊天串流、使用者管理
- YOLO 場地/球員/球體推論
- 回合切割、事件偵測、統計彙整
- vLLM 視覺判斷與聊天問答
- PostgreSQL 紀錄保存
- Docker Compose 部署

不包含：

- 多機分散式任務排程
- 多 GPU 任務調度
- 即時直播分析
- 細緻 RBAC 權限系統

4. 架構設計原則

- 單一影片單一工作空間：每部影片有獨立資料夾，避免輸出互相污染。

- 先完成端到端分析，再逐步優化模組內部分工。
- 分析結果以 JSON 為主體，前端呈現與聊天都以它為資料來源。
- VLM 為可選能力，未啟用時核心分析仍應可運作。
- 優先維持部署簡單度，現階段以 Docker Compose 為主。

5. 部署架構

```

User Browser
  |
  v
Frontend (Next.js, :3000)
  |
  +--> /api/chat proxy
  +--> /api/* rewrite
  |
  v
Backend (FastAPI, :8000)
  |
  +--> PostgreSQL
  +--> YOLO models
  +--> vLLM
  +--> Embedding service
  |
  v
data/ users|guest|videos_chunks|postgres|huggingface

```

Docker Compose 服務分工：

- **frontend**：Next.js UI 與 API proxy
- **backend**：FastAPI 主服務、分析流程、聊天流程
- **postgres**：使用者、影片紀錄、聊天訊息
- **vllm**：多模態聊天與回合判讀
- **vllm-embedding**：RAG 檢索 embedding
- **cloudflared**：外部 HTTPS 入口

6. 邏輯架構

```

[展示層]
Next.js page.tsx
  |- VideoPanel
  |- AnalysisPanel
  |- ChatPanel
  ` - FilePanel

[API 層]
FastAPI routers
  |- video_router
  |- analyze_router
  |- chat_router

```

```
`- user_router

[服務層]
services/analyze
| - video_io.py
| - court.py
| - player.py
| - ball.py
| - buffer.py
| - analysis.py
| - aggregate.py
`- vlm_verify.py

services/chat
| - main.py
| - context.py
| - llm.py
| - tools.py
`- persistence.py

[資料層]
SQLAlchemy + PostgreSQL
| - users
| - analysis_records
`- analysis_messages
```

7. 核心資料流

7.1 影片分析流程

```
上傳影片
-> 分塊合併
-> 必要時轉成瀏覽器相容 H.264
-> 建立 AnalysisRecord 與 session
-> /api/analyze 啟動分析
-> analyze.main.analyze()
-> 場地偵測 / 姿態偵測 / 球追蹤 / buffer 後處理
-> 回合事件與統計彙整
-> VLM 背景判斷回合勝負
-> 輸出 analysis.json / analysis.mp4 / thumbs
-> 前端輪詢 status 並載入結果
```

7.2 聊天流程

```
使用者提問
-> /api/chat
-> ChatService
-> 載入 analysis.json 與歷史訊息
-> embedding 檢索相關回合
```

```
-> 組 system prompt  
-> 呼叫 vLLM 串流輸出  
-> 必要時 tool call 檢視指定回合  
-> SSE 回傳前端  
-> 對話持久化到 analysis_messages
```

8. 模組責任

8.1 前端

- `frontend/app/page.tsx`
 - 管理工作區狀態、分析狀態、當前影片與分頁切換
- `VideoPanel`
 - 上傳、播放、分析啟動、分析結果切換
- `AnalysisPanel`
 - 呈現 rally/player/depth/speed/court 五個分析視圖
- `ChatPanel`
 - 與後端 SSE 串流聊天
- `FilePanel`
 - 歷史紀錄載入與刪除

8.2 後端 API

- `video_router.py`
 - 分塊上傳、影片合併、轉碼、歷史資料載入
- `analyze_router.py`
 - 啟動分析、重新分析、進度查詢
- `chat_router.py`
 - 聊天 SSE 串流端點
- `user_router.py`
 - 註冊、登入、身份查詢

8.3 分析服務

- `main.py`
 - 薄 orchestrator，負責初始化元件與主迴圈
- `video_io.py`
 - 影片 decode/encode
- `court.py`
 - 場地關鍵點與 scene cut
- `player.py`
 - 上下球員姿態偵測
- `ball.py`
 - 球體追蹤、軌跡後處理
- `buffer.py`
 - 滑動窗口、回合緩衝、flush
- `analysis.py`
 - 事件偵測、回合分析、速度與落點推導

- `aggregate.py`
 - 單回合與整體摘要彙整
- `vlm_verify.py`
 - 縮圖輸出與 VLM 輔助判斷

8.4 聊天服務

- `main.py`
 - 聊天主流程與 SSE event 編碼
- `context.py`
 - analysis context 建立與 embedding 檢索
- `llm.py`
 - vLLM API client 與 SSE 解析
- `tools.py`
 - rally clip 相關工具呼叫
- `persistence.py`
 - 訊息持久化

9. 資料設計

9.1 資料庫

- `users`
 - 帳號基本資料
- `analysis_records`
 - 影片與分析任務主紀錄
- `analysis_messages`
 - 聊天訊息歷史

9.2 檔案儲存

```
data/  
  users/{owner_id}/{video_token}/  
    raw.{ext}  
    analysis.mp4  
    analysis.json  
    analysis.log  
    thumbs/  
    rally_embeddings.json  
  
  guest/{video_token}/  
    ...  
  
  videos_chunks/{upload_id}/  
    000000.part  
    000001.part
```

9.3 Session 設計

- 進度與暫時狀態放在 `app.state.session_store`
- DB 負責長期紀錄
- session 過期與 guest 檔案清理由 `lifespan.py` 背景任務處理

這個設計實作簡單，但目前不適合多 backend instance 橫向擴充。

10. 非功能需求

10.1 效能

- 單次分析流程需支援長影片處理
- 聊天需支援 SSE 串流首 token 快速回應
- 影片處理以單機 GPU 為主要執行環境

10.2 可維護性

- 路由、分析、聊天已分模組
- 但 `buffer.py`、`analysis.py`、`aggregate.py` 之間仍需進一步釐清責任邊界

10.3 可部署性

- 以 `docker-compose.yml` 為主
- VLM 透過 profile 啟用，利於資源切分

10.4 安全性

- JWT 身分驗證
- guest token 區分匿名訪客資料
- 影片與輸出路徑需限制在 `DATA_DIR` 下

11. 目前架構觀察

目前程式已具備可運作的主幹，但有幾個值得列為架構改善項目：

1. 文件同步風險
`README.md`、`SPEC.md`、`系統提案.md`、實作行為已有多處重疊，後續容易失真。
2. 分析模組責任邊界需再收斂
`buffer`、`analysis`、`aggregate` 的邏輯分工已有雛形，但仍適合再整理成更清楚的資料契約。
3. `session_store` 為單機記憶體狀態
現階段足夠，但若未來要多實例部署，需改成 Redis 或資料庫型進度儲存。
4. 影音處理策略需明確化
目前轉碼主要是為瀏覽器播放相容性；分析流程與輸出流程是否保留音訊，需要正式定義。
5. 模型比較與方法論尚未文件化收斂
目前 repo 已有大量 ball/court 測試與 artifact，但缺少統一的決策輸出文件。

12. 工作預定

以下排程以「下一階段開發整理」為目的，採 6 週建議排程。若要壓縮成論文或展示版，可將第 4-6 週平行化。

第 1 週：文件與現況對齊

目標：建立單一可信文件基線。

- 完成系統架構文件與規格文件對齊
- 定義文件同步清單
- 補上目前資料流、DB、部署、API 的最小維護規範

交付物：

- 系統架構設計與工作預定.md
- 文件同步規則

驗收標準：

- 主要模組、資料流、資料夾結構與 repo 一致

第 2 週：分析流程重構

目標：降低分析模組耦合。

- 釐清 `buffer.py` / `analysis.py` / `aggregate.py` 的輸入輸出契約
- 明確切出「逐幀處理」、「回合定案」、「摘要彙整」三層
- 補上必要註解與模組邊界文件

交付物：

- 重構後的 analyze service 邏輯分層
- 模組責任說明

驗收標準：

- 主流程可維持既有輸出
- 主要函式輸入輸出更穩定且可測

第 3 週：影音流程修正

目標：修正影片處理策略。

- 將相容性轉碼明確限制為瀏覽器播放需求
- 檢查分析輸出是否保留原始音訊
- 簡化 `backend/services/utils.py:get_video_meta`

交付物：

- 轉碼策略文件
- 音訊保留方案與實作
- `get_video_meta` 簡化結果

驗收標準：

- 不再為 VLM 需求做多餘轉碼
- `analysis.mp4` 音訊策略明確且可驗證

第 4 週：模型與追蹤方法比較

目標：收斂球追蹤方案。

- 比較 `raw`、追蹤器版本、`Finalized` 後處理版本
- 補齊固定測試影片的可視化輸出
- 形成方法選型結論

交付物：

- 方法比較報告
- 測試輸出影片與統計表

驗收標準：

- 至少有一版被明確定為預設方案
- 決策依據可追溯

第 5 週：聊天與檢索穩定化

目標：提高 AI 問答可用性。

- 檢查 SSE 狀態事件與前端呈現
- 驗證 tool calling 對特定回合檢索是否穩定
- 檢查聊天歷史與 record/session 關聯是否一致

交付物：

- 聊天流程測試清單
- 例外狀況處理修正

驗收標準：

- 已分析與未分析影片都能正確回應
- tool call 流程可重現

第 6 週：發版整理

目標：準備展示或正式交付。

- 補齊 README / SPEC / 架構文件同步
- 清理過期實驗記錄與不必要說明
- 建立最小回歸測試清單

交付物：

- 可交付版本文件集
- 發版檢查表

驗收標準：

- 新成員能依文件完成部署與基本驗證

13. 優先級排序

建議優先順序如下：

1. 文件對齊與架構邊界確認
2. 分析服務重構
3. 影音流程修正
4. 模型比較定案
5. 聊天穩定化
6. 發版整理

14. 風險與對策

風險	說明	對策
文件與實作脫節	文件多且分散	建立同步清單，重大修改同步更新
分析模組耦合過深	後續難改、難測	先定義資料契約再拆分
記憶體型 session 無法擴充	多實例時進度會失真	後續改 Redis
VLM 資源成本高	GPU 佔用大	保留可關閉設計，核心分析不依賴 VLM
模型比較結果分散	難形成產品決策	產出統一比較報告與預設方案

15. 建議後續文件分工

為避免之後再次混亂，建議保留以下定位：

- README.md
 - 給新使用者的快速理解、安裝與執行
- SPEC.md
 - 系統規格、API、資料格式
- 系統提案.md
 - 報告或對外提案敘述
- 待做工作.md
 - 臨時工作筆記與零散待辦
- 系統架構設計與工作預定.md
 - 架構現況與近期工作排程基線

16. 結論

目前專案已具備完整的端到端主流程，包含上傳、分析、視覺化與 AI 問答，架構上已經足以支撐展示與持續開發。下一階段的重點不是再擴功能，而是把文件、模組邊界、影音策略與模型決策收斂，讓系統從「可運作」提升到「可維護、可驗證、可交付」。